

Build & Deployment Audit — H.A.M.M.E.R. POS/ERP

Fecha: 2026-05-12
Auditor: DevOps / Full-Stack Senior (Agente Automatizado)
Branch: production-feature-security-integrity-hardening-ux
Versión: 0.2.0

1. Resumen Ejecutivo

Métrica	Resultado
Estado General	✅ LISTO PARA PRODUCCIÓN (con warnings menores)
Errores TypeScript	0 — compila limpio con strict: true
Warnings de Build	2 (ambientales, no bloqueantes)
Vulnerabilidades npm	2 moderate (postcss dentro de next, no re-mediable sin breaking change)
Build Exitoso	✅ Sí (next build completa sin errores)
Tamaño del Build	270 MB (.next/)
Lint	⚠️ No configurado (sin eslint)
Tests E2E	⚠️ Requieren infraestructura (Playwright + DB + seed)

2. Cadena de Validación

2.1 Prisma Generate

```
✅ EXITO
Prisma schema loaded from prisma/schema.prisma
✅ Generated Prisma Client (v6.19.3) to ./node_modules/@prisma/client in 219ms
```

Warning (baja): package.json#prisma está deprecado, migrar a prisma.config.ts antes de Prisma 7.

2.2 TypeCheck (`npx tsc --noEmit`)

✅ ÉXITO – 0 errores TypeScript

Configuración: `strict: true`, `target ES2022`, `skipLibCheck: true`.

2.3 Lint

⚠️ NO CONFIGURADO – No existe script "lint" ni archivos `.eslintrc` / `eslint.config.*`

Recomendación (media): Agregar ESLint con `next/core-web-vitals` y `typescript-eslint`.

2.4 Build (`npm run build`)

✅ ÉXITO – Compiled successfully in 10.9s

- Next.js 15.5.18 (actualizado tras `npm audit fix`)
- Todas las rutas compiladas correctamente
- Incluye: 60+ rutas API, 30+ páginas de la app, endpoints `/health` y `/ready`
- **Requisito:** Variables `DATABASE_URL` y `AUTH_SESSION_SECRET` necesarias en build-time (el Dockerfile ya las provee como `ARG` / `ENV` de build)

2.5 Tests

⚠️ NO EJECUTADOS – Requieren: PostgreSQL, seed de datos, Playwright con Chromium

El proyecto tiene tests E2E configurados con Playwright y docker-compose para ejecución completa. Comando: `docker compose up e2e` o `npm run quality:gate` (requiere infra completa).

3. Errores Encontrados y Corregidos

TypeScript Errors

Archivo	Línea	Error	Corrección
(ninguno)	—	—	TypeScript compila limpio, 0 errores

No se requirieron correcciones TypeScript. El proyecto compila sin errores con `strict: true`.

Justificación de `any`

No se identificaron usos de `any` que requieran justificación — el modo estricto está activo y no hay errores implícitos.

4. Configuración

4.1 package.json

Scripts

Script	Existe	Notas
build	✓	npm run prisma:generate && next build
start	✓	next start
dev	✓	next dev
start:railway	✓	next start --hostname 0.0.0.0 --port \${PORT}
prisma:generate	✓	
prisma:migrate:deploy	✓	
typecheck	✓	tsc --noEmit
lint	✗	No existe — recomendación agregar
quality:gate	✓	Cadena completa de valida- ción
env:validate	✓	Validación de variables de entorno
seed / seed:demo / seed:production	✓	

Engines

```
"engines": { "node": ">=22 <23", "npm": ">=11 <12" }
```

Warning (baja): El npm actual del ambiente (10.9.2) no cumple `>=11 <12`, pero funciona correctamente. En CI/CD asegurar npm 11+.

Versiones Críticas

Paquete	Versión Instalada	Estado
next	15.5.18	✓ Actualizado (tras npm audit fix)
prisma	6.19.3	✓
@prisma/client	6.19.3	✓ Sincronizado con prisma
react	19.1.0	✓ Compatible con Next.js 15
react-dom	19.1.0	✓ Sincronizado
typescript	^5.8.2	✓
zod	^3.24.3	✓

4.2 Dockerfile

Análisis

Criterio	Estado	Detalle
Multi-stage build	✓	4 stages: base → deps → builder → runtime
Node version correcta	✓	node:22-alpine
npm ci en vez de npm install	✓	Tanto en deps como en runtime
Prisma generate en build stage	✓	Incluido en npm run build
Prisma migrate en runtime	✓	Vía entrypoint.sh
Variables build-time	✓	BUILD_DATABASE_URL y BUILD_AUTH_SESSION_SECRET como ARG
Puerto expuesto	✓	EXPOSE 3000
Entrypoint robusto	✓	set -Euo pipefail, validación de env, migraciones
Usuario no-root	✗	Falta — ejecuta como root
HEALTHCHECK	✗	Falta en Dockerfile (Railway lo tiene en railway.json)

Recomendaciones Dockerfile

```
# Agregar antes de EXPOSE en el stage runtime:
RUN addgroup --system --gid 1001 nodejs \
  && adduser --system --uid 1001 nextjs
USER nextjs

# Agregar HEALTHCHECK:
HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1
```

4.3 docker-compose.yml

Criterio	Estado
Servicio PostgreSQL con healthcheck	✓
Volumen persistente para DB	✓
Variables de entorno configuradas	✓
Servicio E2E con Playwright	✓
Servicio release-check	✓
⚠ AUTH_SESSION_SECRET hardcoded	⚠ Usar <code>.env</code> externo en producción

4.4 railway.json

```
{
  "build": { "builder": "DOCKERFILE" },
  "deploy": {
    "startCommand": "npm run start:railway",
    "healthcheckPath": "/health",
    "healthcheckTimeout": 300,
    "restartPolicyType": "ON_FAILURE",
    "restartPolicyMaxRetries": 10
  }
}
```

✓ Configuración correcta para Railway.

Nota: El `startCommand` en `railway.json` puede sobrescribir el CMD del Dockerfile.

Las migraciones se ejecutan en el entrypoint, asegurar que Railway use el ENTRYPOINT del Dockerfile.

4.5 Entrypoint (`docker/entrypoint.sh`)

Criterio	Estado
<code>set -Euo pipefail</code>	✓ Fail-fast habilitado
Validación de env (strict)	✓
Prisma generate	✓
Prisma migrate deploy (producción)	✓ Condicional a <code>RUN_MIGRATIONS=true</code>
Logging con timestamps	✓

5. Variables de Entorno

Requeridas en Producción

Variable	Tipo	Obligatoria	Notas
<code>DATABASE_URL</code>	string (URL)	✓	<code>postgresql:// user:pass@host:5432/ db</code>
<code>AUTH_SESSION_SECRET</code>	string (≥32 chars)	✓	<code>openssl rand -hex 32</code>
<code>NODE_ENV</code>	enum	✓	Debe ser <code>production</code>
<code>APP_ENV</code>	enum	✓	Debe ser <code>production</code>

Opcionales

Variable	Default	Notas
<code>PORT</code>	3000	Puerto del servidor
<code>AUTH_SESSION_TTL_HOURS</code>	12	Duración de sesión
<code>RUN_MIGRATIONS</code>	true	Ejecutar migraciones al arran- car
<code>EN- ABLE_CASH_CLOSURE_SCHEDULER</code>	false	Scheduler de cierre de caja

Documentación

- ✓ `.env.example` — completo para desarrollo
- ✓ `.env.production.example` — completo para producción con instrucciones claras
- ✓ `scripts/validate-env.mjs` — validación exhaustiva con modo estricto

6. Vulnerabilidades

npm audit (post npm audit fix)

Paquete	Severidad	Descripción	Remediable
postcss <8.5.10 (dentro de next)	Moderate	XSS via CSS Stringify Output	❌ Solo con --force (breaking change a next 9.x)
next transitive postcss	Moderate	Misma vuln, dependencia interna	❌ Esperar fix upstream

Veredicto: No hay vulnerabilidades **high** o **critical** pendientes. Las 2 moderate son internas a Next.js y no explotables directamente en un POS/ERP server-rendered.

7. Warnings Críticos

#	Warning	Severidad	Acción
1	EBADENGINE : npm 10.9.2 no cumple >=11 <12	Baja	Funcional; actualizar npm en CI/CD
2	Prisma package.json#prisma deprecado	Baja	Migrar a prisma.config.ts antes de Prisma 7
3	Sin ESLint configurado	Media	Agregar next lint con reglas TypeScript
4	Dockerfile sin usuario no-root	Media	Agregar USER nextjs en stage runtime
5	Dockerfile sin HEALTHCHECK	Baja	Railway ya tiene healthcheck; agregar en Docker para otros entornos
6	AUTH_SESSION_SECRET hardcoded en docker-compose.yml	Media	Usar archivo .env externo en producción

8. Checklist de Deploy

Pre-Deploy

- ☒ npm audit sin vulnerabilidades críticas/high
- ☒ TypeScript sin errores (0 errores, strict mode)
- ☒ Build exitoso (`next build` completa)
- ☐ Tests E2E pasando (requiere infra: `docker compose up e2e`)
- ☐ Variables de entorno configuradas en plataforma de deploy
- ☐ `DATABASE_URL` apuntando a PostgreSQL de producción
- ☐ `AUTH_SESSION_SECRET` generado con `openssl rand -hex 32`
- ☐ `NODE_ENV=production` y `APP_ENV=production`

Deploy

- ☐ Ejecutar migraciones: `npx prisma migrate deploy` (automático vía entrypoint)
- ☐ Verificar seed inicial si es primera vez: `npm run seed:production`
- ☐ Healthcheck respondiendo: `GET /health` → 200
- ☐ Readiness check: `GET /ready` → 200
- ☐ Logs sin errores

Post-Deploy

- ☐ Login funciona con credenciales
- ☐ POS funciona (crear venta, cobrar)
- ☐ Operaciones CRUD (productos, categorías, usuarios)
- ☐ Reportes y analytics cargan
- ☐ Monitoreo activo (logs, métricas)

9. Comandos Finales

Build Local

```
# Instalar dependencias
npm ci

# Generar cliente Prisma
npm run prisma:generate

# Verificar tipos
npm run typecheck

# Build de producción (requiere DATABASE_URL y AUTH_SESSION_SECRET)
DATABASE_URL="postgresql://user:pass@localhost:5432/hammer" \
AUTH_SESSION_SECRET="$(openssl rand -hex 32)" \
npm run build
```

Deploy Docker

```
# Build de imagen
docker build -t hammer-pos .

# Ejecutar con variables de entorno
docker run -p 3000:3000 \
  -e DATABASE_URL="postgresql://user:pass@db:5432/hammer" \
  -e AUTH_SESSION_SECRET="$(openssl rand -hex 32)" \
  -e NODE_ENV=production \
  -e APP_ENV=production \
  hammer-pos
```

Deploy Docker Compose (desarrollo/staging)

```
docker compose up -d db app
```

Deploy Railway

```
# Variables requeridas en Railway dashboard:
# DATABASE_URL, AUTH_SESSION_SECRET, NODE_ENV=production, APP_ENV=production

railway up
```

Ejecutar Tests E2E

```
docker compose up e2e
```

Quality Gate Completo

```
docker compose run --rm release-check
```

10. Archivos Modificados en esta Auditoría

Archivo	Cambio
package-lock.json	Actualizado por npm audit fix (next 15.5.15 → 15.5.18)
BUILD_AUDIT_REPORT.md	Creado — este reporte

Nota: No se modificaron archivos de código fuente. El proyecto compiló limpio sin necesidad de correcciones TypeScript.

11. Veredicto Final

✓ LISTO PARA PRODUCCIÓN (con warnings menores)

El proyecto H.A.M.M.E.R. POS/ERP **compila exitosamente** con TypeScript strict mode, 0 errores de tipos, y un build de Next.js 15.5.18 funcional. La arquitectura de deployment (Dockerfile multi-stage, entrypoint robusto, validación de env, railway.json) está bien diseñada.

Bloqueadores

Ninguno. El proyecto puede desplegarse tal como está.

Riesgos Aceptables

1. **postcss moderate vuln** — Interna a Next.js, sin vector de ataque directo en server-rendered app
2. **Sin ESLint** — No impide el funcionamiento pero reduce la calidad de código a largo plazo
3. **Dockerfile ejecuta como root** — Funcional pero no es best practice de seguridad

Próximos Pasos (recomendados, no bloqueantes)

1. **Agregar usuario no-root en Dockerfile** — `USER nextjs` en stage runtime
2. **Agregar HEALTHCHECK en Dockerfile** — Para entornos fuera de Railway
3. **Configurar ESLint** — `npx next lint --init` + reglas TypeScript
4. **Migrar config Prisma** — De `package.json#prisma` a `prisma.config.ts`
5. **Ejecutar tests E2E** — Validar funcionalidad completa antes del primer deploy
6. **Monitorear** — Configurar alertas para `/health` y logs de errores
7. **Actualizar npm** — A versión 11+ para cumplir con engines declarados